



Faculty of Engineering and Applied Science

Department of Electrical, computer and Software Engineering

SOFE 3650 – Software Design & Architecture

Deliverable 3

Due Date: December 3rd, 2025

Course Instructors: Dr Hani Sami

CRN: 45740 - Group # 8

Student Name	Student Number
Christopher Lui	100912564
Geraline Chavez	100890130
Nicholas Furtado	100908880

1.4 ADD Iteration 3: Addressing Quality Attribute Scenario Driver

This iteration focuses on the QA-4 quality attribute scenario defined earlier in the architecture design process.

This scenario addresses Privacy and Security by ensuring that when a user attempts to access unauthorized academic information, the system denies access and logs the attempt to comply with institutional privacy policies.

The inputs relevant to this iteration include:

- The selected **Quality Attribute Scenario 4** (Privacy & Security).
- Previously defined architectural constraints.
- Existing components identified in Iterations 1 and 2 (AUTH SSO Provider, API Gateway, User/Profile Service, Monitoring Component).
- Functional requirements related to access control and identity validation.

1.4.1 Step 2: Establish Iteration Goal by Selecting Drivers

In this iteration, the architect chooses to focus on Quality Attribute Scenario 4: *When a user tries to access unauthorized academic information, the system denies access and logs the attempt, enforcing institutional privacy policies.*

1.4.2 Step 3: Choose One or More Elements of the System to Refine

For this scenario, the elements to refine are the ones related to authentication and security:

- AUTH SSO Provider
- API Gateway (Ingress)
- User/Profile Service
- Monitoring Component

1.4.3 Step 4: Choose One or More Design Concepts That Satisfy the Selected Drivers

The design concepts used in this iteration are the following:

Design Decisions & Location	Rationale & Assumptions
Introduce authorization/access control to ensure that only permitted users can access sensitive information	Prevents users with unauthorized roles from accessing higher-level information, such as grades and exam details. In addition, ensures that certain data can only be accessed by the owner and those they choose to share it with, such as the lecturer and a student's grades.
Introduce audit logging to	By actively logging all actions committed by each user to

monitor all activity, including unauthorized attempts	the monitoring component, attempts to access prohibited information can be quickly traced back to the perpetrator.
---	--

1.4.4 Step 5: Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces

The instantiation design decisions are summarized in the following table:

Design Decisions & Location	Rationale & Assumptions
Apply role-based authorization and access control	Use the AUTH SSO provider to authenticate users and assign identity tokens. The API gateway must validate these tokens to allow permitted users to access privatized information. This ensures that users without the correct identity tokens are rejected, while recording the attempt.
Define clear-cut roles with varying levels of access	Provide a profile service that stores user attributes, roles and permissions. The layered architecture system can take advantage of this and maintain security between different types of accounts.
Use the API Gateway to monitor all requests and track them in an audit log	Allow the gateway to keep track of all activities that pass through, writing them to a log and pushing them to an external monitoring component, such as CloudWatch. This allows administrators to easily keep track of malicious attempts to access prohibited information.

The results of these instantiation decisions are recorded in the next step.

1.4.5 Step 6: Sketch Views and Record Design Decisions

The following figure displays a refined deployment diagram that includes the incorporation of the security features. It explicitly includes monitoring components that ensure authorization is enforced and audit trails are included. The diagram illustrates how the SSO authenticates users, the API gateway validation access tokens and how requests, especially unauthorized requests are logged within the system.

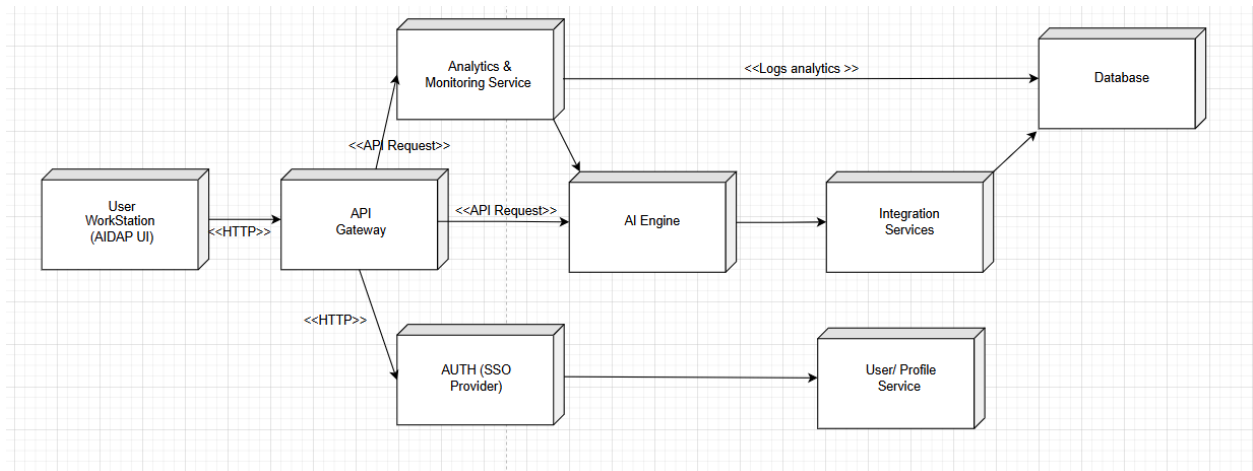


Figure # - Refined Deployment diagram

The following table describes responsibilities for elements that have not been previously listed (in past iterations):

Element	Responsibility
AUTH (SSO Provider)	Checks users' institutional credentials to generate (identity) tokens. The tokens have the user's account ability encoded within, providing permissions to the account.
API Gateway	Acts as the central entry point for all client requests into the system, handling all security and routing. The gateway validates users tokens from the SSO and enforces the permissions applicable to the account.
Analytics & Monitoring Service	Provides monitoring services and audits/ logs all events that occur into the database
User Profile Service	User account management & profile related information (i.e. personal information, transcript, etc).

The UML sequence diagram shown in figure # illustrates how the API gateway enforces permission abilities to support UC - 2 (Maintaining student privacy) which is associated with QA - 4 (Privacy & Security). It shows how the system checks users credentials, and generates a token that is passed between different system components to verify the users ability to access specific features and information.

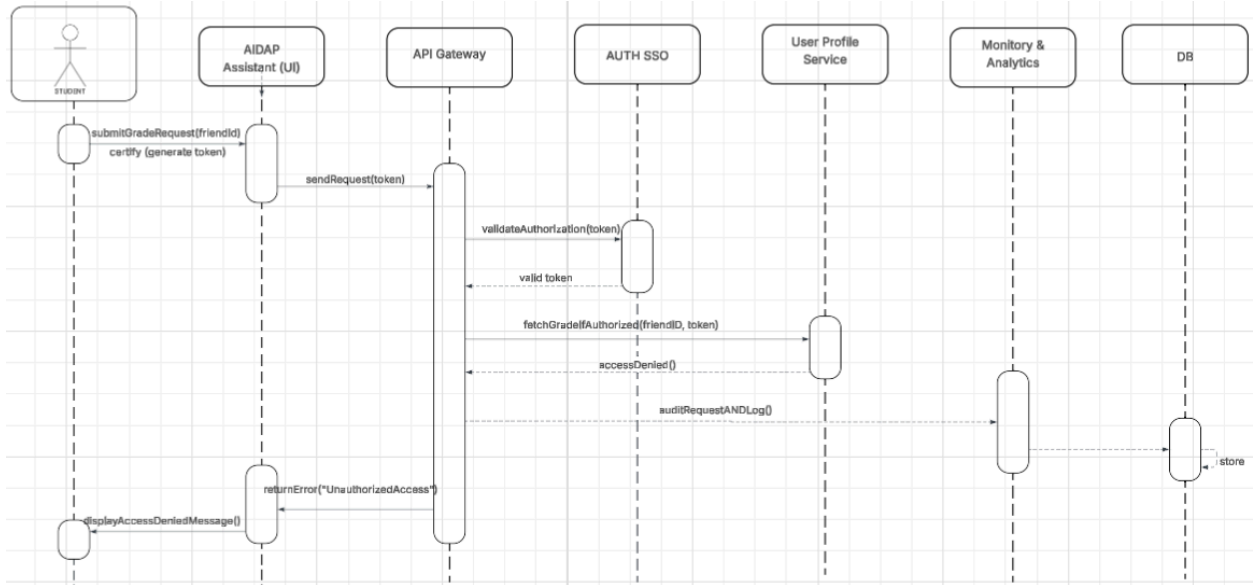


Figure # Sequence Diagram of UC - 2

From the interactions presented and identified in the sequence diagram above, initial methods for interfaces of all the interacting elements present can be identified:

Method Name	Description
Element: AIDAP Assistant (UI)	
submitGradeRequest(friendId)	Student submits request to view their friends grades
displayAccessDeniedMessage()	User receives a prompt stating their access is denied
certify(token)	Users credentials
Element: API Gateway	
sendRequest(token)	Passes request information & credentials of user making the request
returnError("UnauthorizedAccess")	Returns authorization failure
Element: AUTH SSO	
validateAuthorization(token)	Compares and determines if logged in account and requested account are equal or vary
Element: User Profile & Services	

fetchGradeIfAuthorized(friendID, token) accessDenied()	Attempts to fetch grade if credentials are equal, if not it denies access to information Replies with a denied access
Element: Monitoring & Analytics	
auditRequestANDLog()	Passes users request to DB to be logged and stored

1.4.6 Step 7: Perform Analysis of Current Design and REview Iteration Goal and Achievement of Design Purpose

In the following iteration, design decisions have been made to address QA -4. The following table summarizes the status of the different drivers and the decisions that were made during the iteration. Drivers that were completely addressed in the period's iteration have been removed from the table.

Not Addressed	Partially Addressed	Completely Addressed	Design Decisions Made During the Iteration
	UC - 2		Required modules that support Student privacy have been identified.
	QA - 4		Security & Privacy partially addressed through the user authentication and audit process to help aid in denying unauthorized access.
	CRN - 2		The architectural layout of the system incorporates integrated protection practices, partially addressing the security concern with AI sensitive information responses.
	CRN - 7		Additional modules were identified, separating the user history and account information into its own location.
		CON - 4	Logically structured architecture with clearly defined layers separating the users information into its own module, with API gateway ensuring history is only accessible to the user with the correct login

			credentials (Token generated by SSO).
--	--	--	---------------------------------------

Project assessment- ATAM

Scenario Analysis

Scenario 1 - SEC–S1: Student attempts to view another student’s grades

Element	Description
Attribute	Security & Privacy
Stimulus	A logged-in student attempts to access another student's grade record.
Environment	Normal system operation; user is authenticated but unauthorized for the resource.
Response	API Gateway checks RBAC rules using SSO token, detects insufficient permissions, blocks request, and returns no grade data.

Scenario 2 - SEC–S2: Multiple unauthorized access attempts occur

Element	Description
Attribute	Security
Stimulus	A user repeatedly attempts to access a protected resource.
Environment	High backend activity during peak usage.
Response	API Gateway rejects each request and logs all attempts into the Monitoring Component.

Scenario 3 -SEC–S3 / MOD–S1: Admin updates user roles or permissions

Element	Description
Attribute	Modifiability & Security

Stimulus	Admin modifies a student's or instructor's roles or permissions.
Environment	Normal system operation with active users.
Response	User/Profile Service updates permissions; API Gateway applies new rules on the next request.

Scenario 4 -PERF-S1: High volume of login and token validation requests

Element	Description
Attribute	Performance
Stimulus	Many students try to log in simultaneously.
Environment	Heavy concurrent load on authentication services.
Response	SSO generates signed tokens efficiently; API Gateway validates tokens quickly without blocking.

Scenario 5 - PERF-S2: High volume of academic data retrieval requests

Element	Description
Attribute	Performance & Security
Stimulus	Students repeatedly request grades, history, or participation data.
Environment	High traffic from multiple front-end clients.
Response	API Gateway reuses signed tokens, routes requests efficiently, and denies unauthorized requests early.

1. ATAM risk assessment table

I D	Scenario	Quality Attribute	Architectural Decision	Sensitivit y	Tradeof f	Ris k	Non- Risk
1	SEC–S1: student attempts to view another student's grades	Security & Privacy	AD1 – API Gateway enforces RBAC using SSO token	S1	T1	R1	NR1
2	SEC–S1 / SEC–S2	Security & Auditability	AD3 – Gateway logs all requests to Monitoring Component	S2	T2	R2	NR2
3	SEC–S3 & MOD–S1	Modifiability & Security	AD2 – Roles & permissions stored in User/Profile Service	S3	T3	R3	NR3
4	PERF–S1 / PERF–S2	Performance & Security	AD4 – SSO issues signed identity tokens reused across requests	S4	T4	R4	NR4
5	SEC–S1 / SEC–S2 / PERF–S2	Security & Performance	AD5 – Unauthorized requests denied at API Gateway and logged	S5	T5	R5	NR5

2. Description of the risks, non-risks, sensitivity and tradeoffs.

2.1 Risks (R)R1. Incorrect or incomplete authorization rules in the API Gateway may allow unauthorized access to private academic information.

R2. High volumes of access attempts may overload the logging process, causing missing entries in the Monitoring Component.

R3. Outdated or inconsistent role/permission data in the User/Profile Service may grant users more privileges than intended.

R4. Token lifetime settings may cause performance issues (too frequent re-authentication) or security vulnerabilities (tokens valid for too long).

R5. If access checks are performed too late in the pipeline, partial information may be exposed before the request is denied.

2.2 Non-Risks (NR)

NR1. Centralizing access control in the API Gateway ensures consistent enforcement of security rules across all services.

NR2. Authorization is kept separate from the UI and AI Engine logic, preventing bypass through client-side behaviour or prompts.

NR3. Unauthorized requests are blocked at the gateway even if the Monitoring Component is temporarily unavailable.

NR4. Using signed identity tokens reduces the need to send credentials repeatedly, lowering the risk of credential leakage.

NR5. The layered architecture and independent services keep privacy-critical functions isolated from unrelated components.

2.3 Sensitivity Points (S)

S1. The accuracy of authorization rules in the API Gateway directly determines whether private data is protected.

S2. The Monitoring Component's capacity influences how reliably unauthorized access attempts are recorded.

S3. The role and permission structure in the User/Profile Service strongly affects both security enforcement and ease of updates.

S4. Token expiration and signing parameters impact the balance between secure authentication and response time.

S5. Logging granularity (what is logged and how often) impacts system overhead and the usefulness of security records.

2.4 Tradeoffs (T)

T1. Stronger authorization checks improve security but increase request processing time in the API Gateway.

T2. Detailed logging improves incident analysis but increases storage usage and processing overhead.

T3. A flexible and expressive role model improves maintainability but increases configuration and testing complexity.

T4. Short-lived tokens improve security but require more frequent validation, potentially affecting system performance.

T5. Centralizing access control in the API Gateway simplifies policy enforcement but creates a potential bottleneck under high load.

3. ATAM Utility tree

Utility

├── **Security & Privacy (H, H)**

| ├── **SEC-S1 (H, H):**

| | **Deny unauthorized access to academic records**

| | **and log the attempt.**

| └── **SEC-S2 (H, M):**

| **Ensure denied attempts are tied to identity**

| **and timestamp reliably.**

|

├── **Performance (M, M)**

| ├── **PERF-S1 (H, M):**

| | **Token validation + access checks must**

| | **complete within acceptable response time.**

| └── **PERF-S2 (M, H):**

| **High-volume academic data requests**

| **must not degrade system responsiveness.**

|

└─ **Modifiability (M, M)**

└─ **MOD-S1 (H, M):**

| **Adding new roles/permissions should only**
| **require updates in Profile Service + Gateway.**

└─ **MOD-S2 (M, M):**

Changing monitoring backend must not
 affect request-handling logic.

(H = High priority, M = Medium priority, L = Low effort)